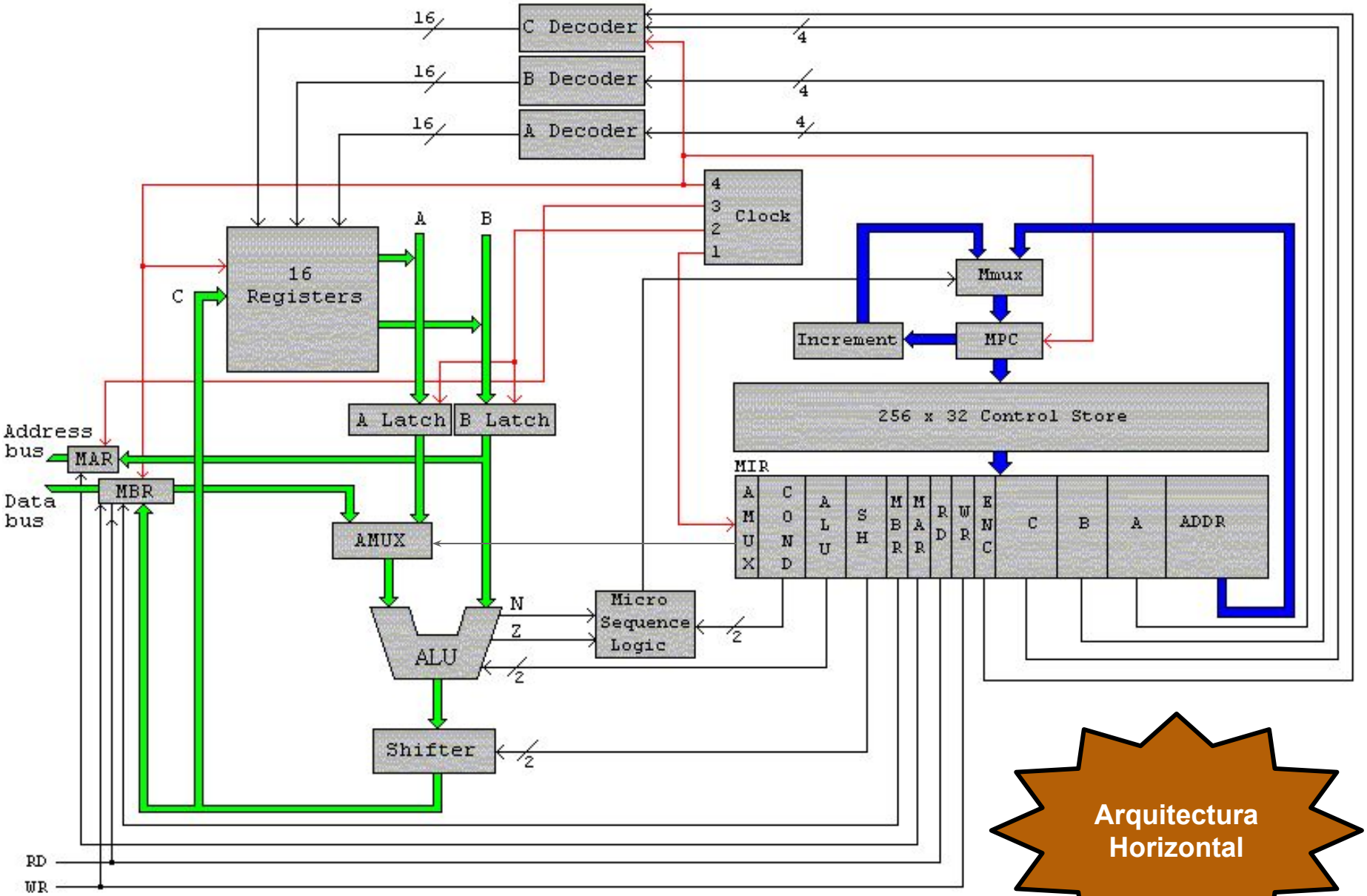
Micro Arquitectura Vertical

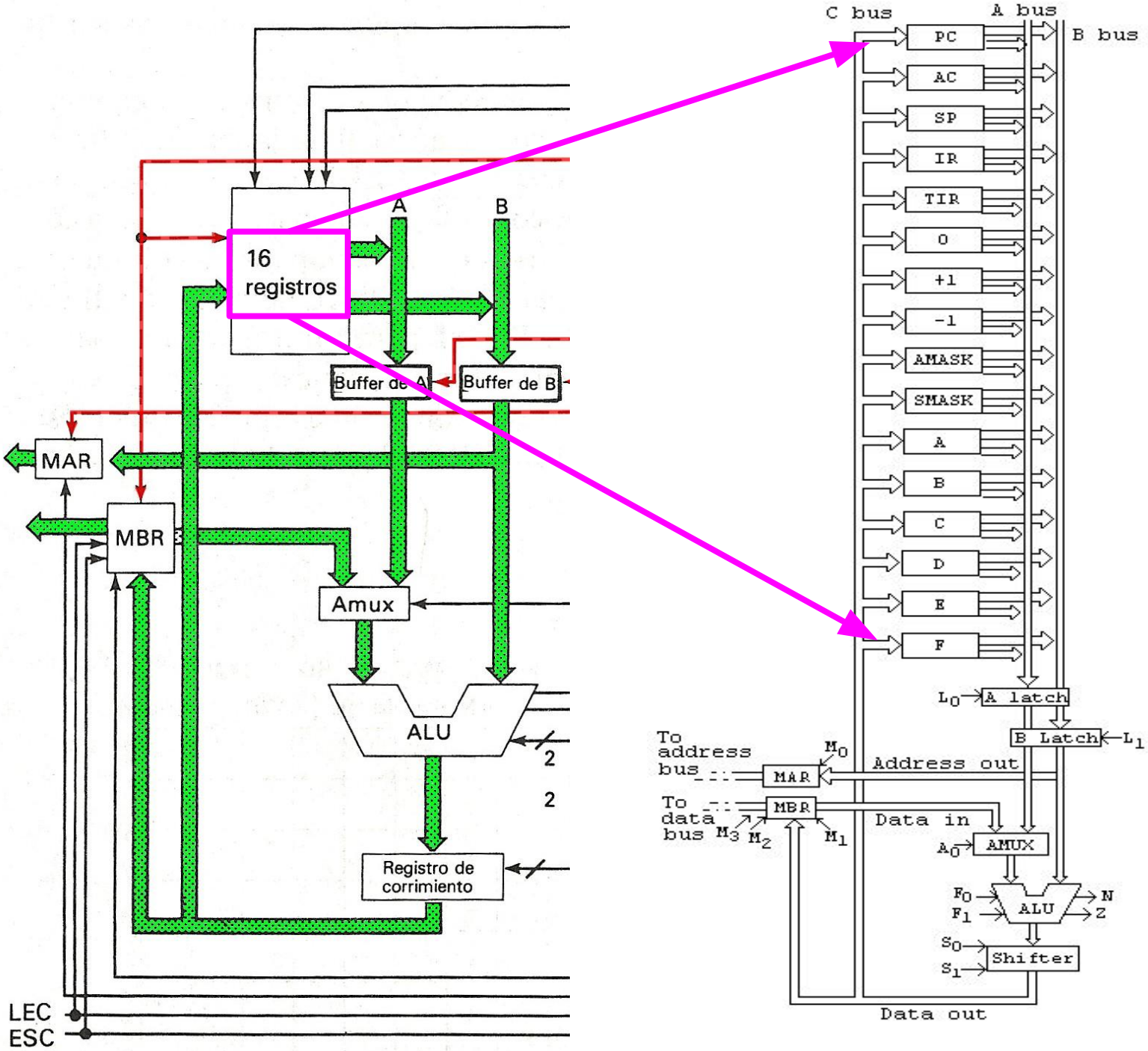
Nano-Programación y Pipe-Line

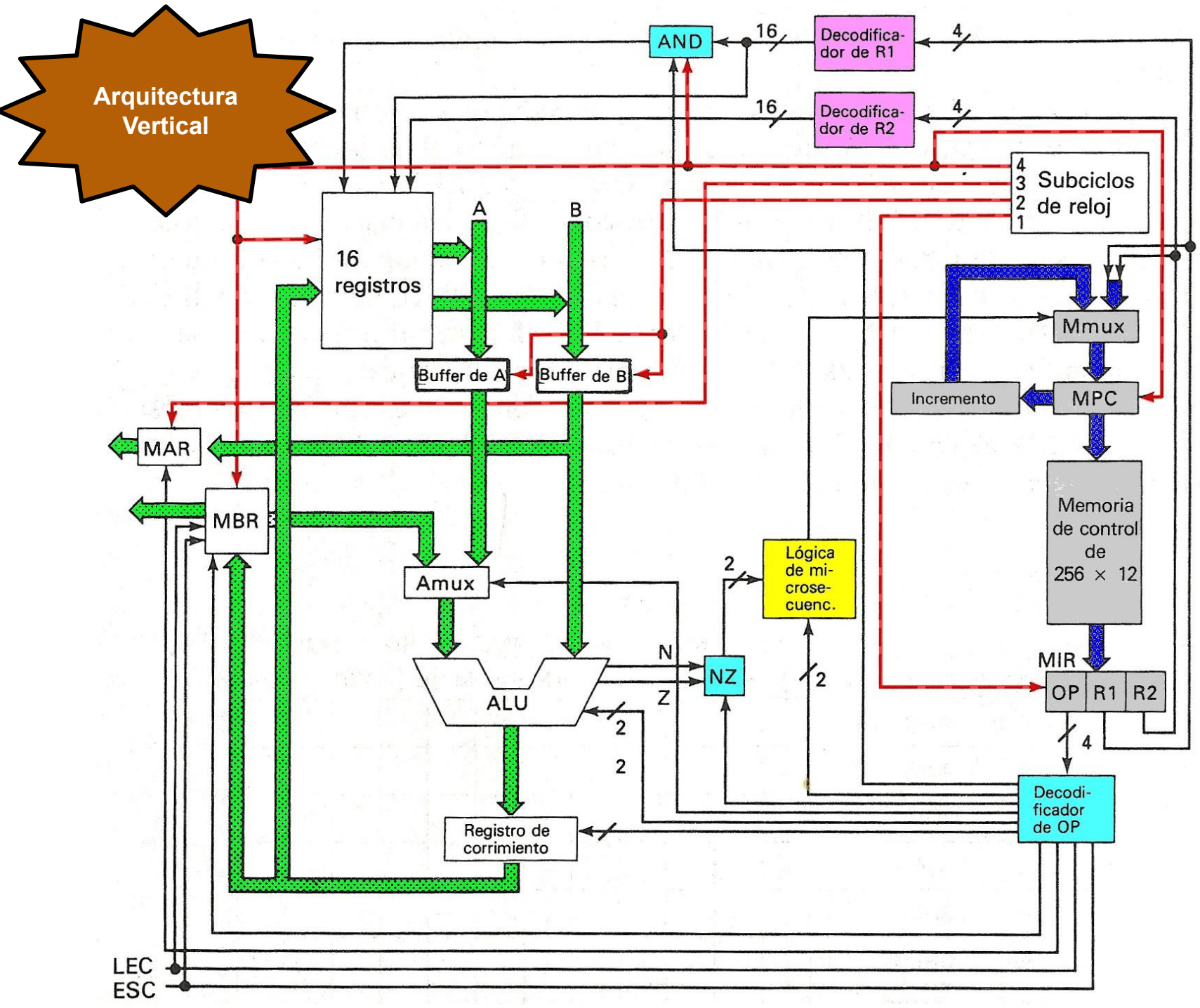
Arquitectura de computadoras

Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta

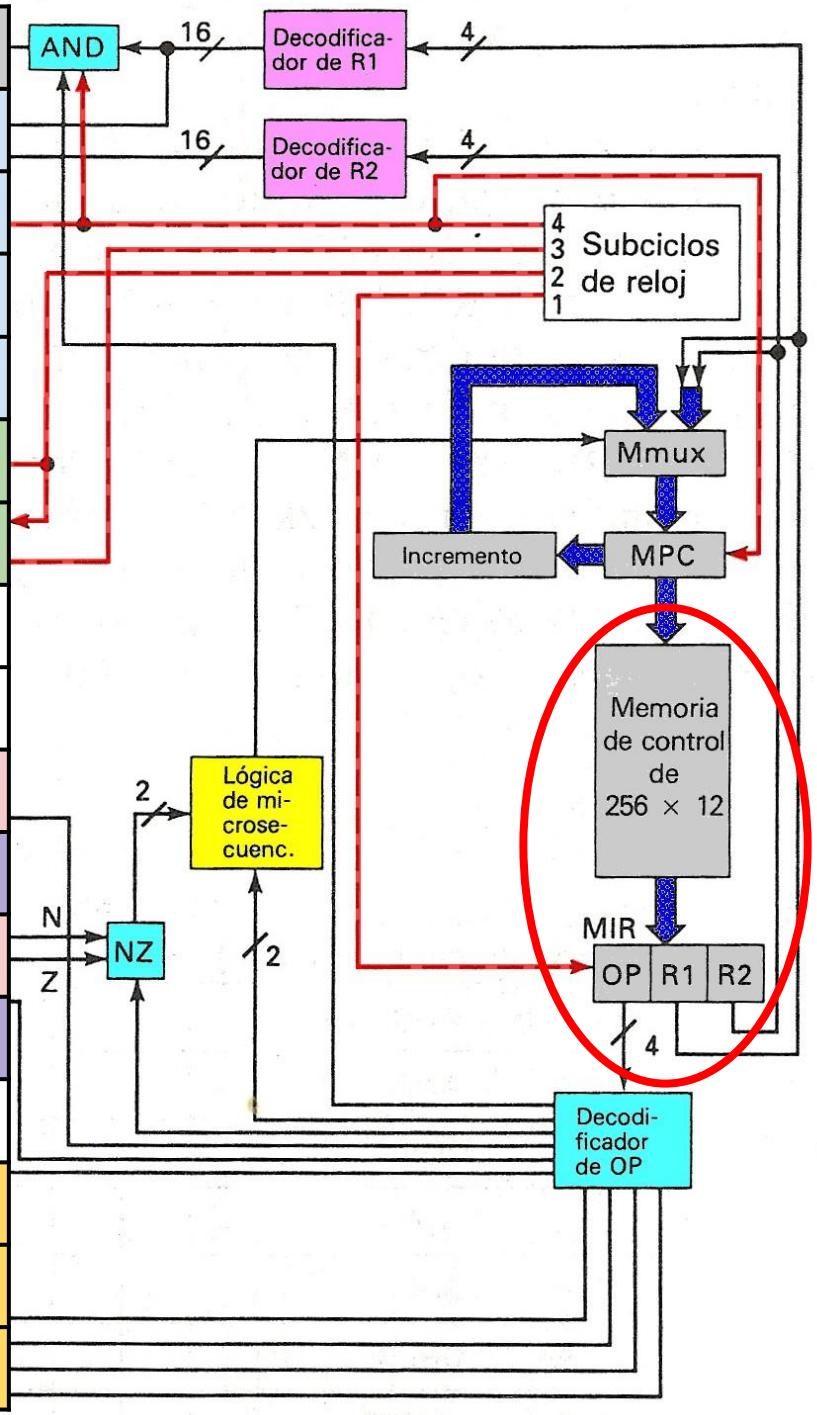


Arquitectura Horizontal


16 registros
A
B
Buffer de A
Buffer de B
MAR
MBR
Amux
ALU
2
2
Registro de corrimiento
LEC
ESC
C bus
A bus
B bus
PC
AC
SP
IR
TIR
0
+1
-1
AMASK
SMASK
A
B
C
D
E
F
L_0
A latch
B Latch
L_1
To address bus
M_0
MAR
Address out
To data bus
MBR
M_3
M_2
M_1
Data in
A_0
AMUX
F_0
F_1
ALU
N
Z
S_0
S_1
Shifter
Data out

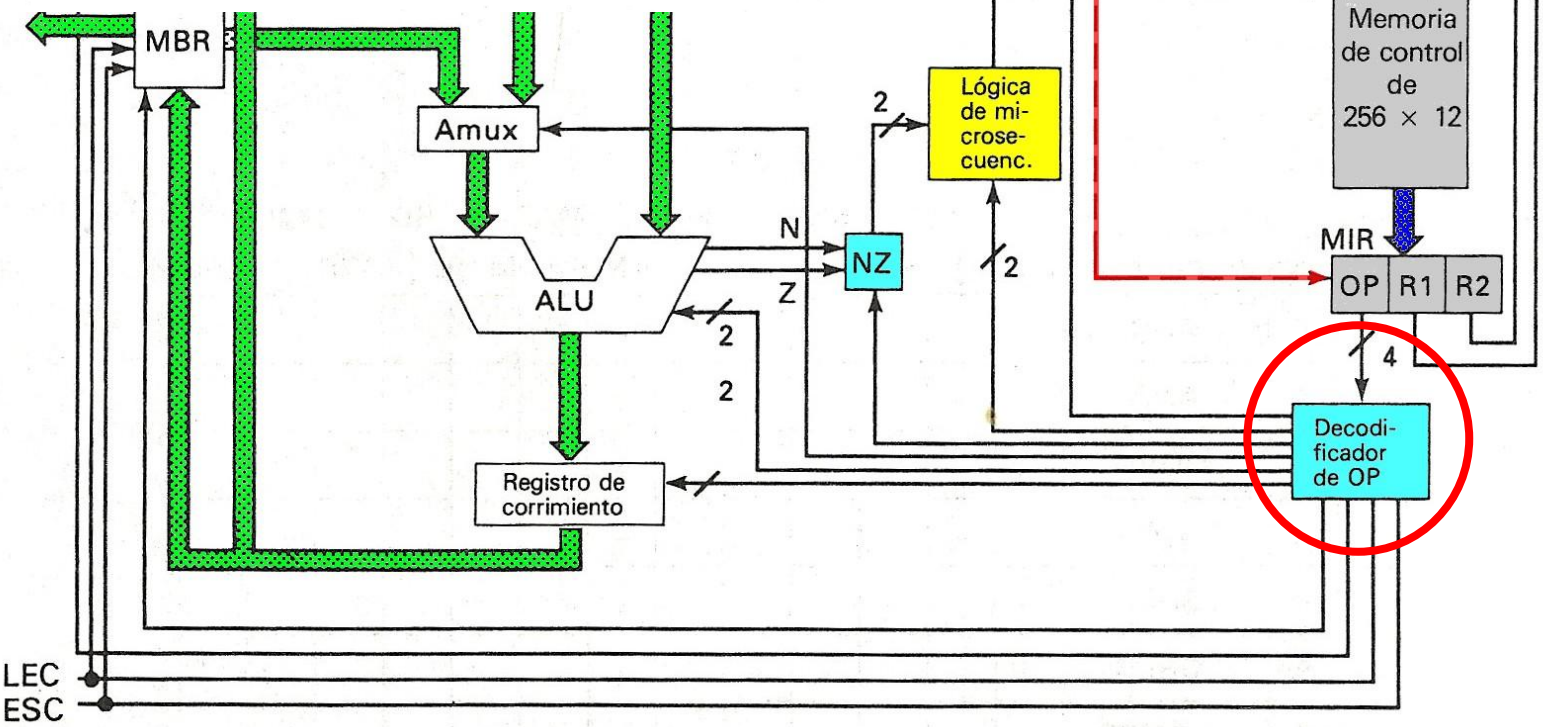

Arquitectura Vertical
AND
16
Decodificador de R1
4
16
Decodificador de R2
4
4
3
2
1
Subciclos de reloj
16 registros
A
B
Buffer de A
Buffer de B
Mmux
Incremento
MPC
MAR
MBR
Memoria de control de 256 × 12
Amux
2
Lógica de microsecuenc.
N
NZ
2
ALU
Z
MIR
OP
R1
R2
2
2
4
Decodificador de OP
Registro de corrimiento
LEC
ESC

OP	Mnemónico	Instrucción
0000	ADD	Suma
0001	AND	Y bit a bit
0010	MOVE	Mueve registro
0011	COMPL	Complemento
0100	LSHIFT	Desplaza a la Izquierda
0101	RSHIFT	Desplaza a la Derecha
0110	GETMBR	Almacena MBR en registro
0111	TEST	Examina registro
1000	BEGRD	Comienza la Lectura
1001	BEGWR	Comienza la Escritura
1010	CONRD	Continúa la Lectura
1011	CONWR	Continúa la Escritura
1100		
1101	NJUMP	Salta si N=true
1110	ZJUMP	Salta si Z=true
1111	UJUMP	Salta siempre



Binario	Mnemónico	Instrucción	Significado
0000	**ADD**	Suma	**r1:=r1+r2**
0001	**AND**	Y bit a bit	**r1:=r1&r2**
0010	**MOVE**	Mueve registro	**r1:=r2**
0011	**COMPL**	Complemento	**r1:=inv(r2)**
0100	**LSHIFT**	Desplaza a la Izquierda	**r1:=r2<<1**
0101	**RSHIFT**	Desplaza a la Derecha	**r1:=r2>>1**
0110	**GETMBR**	Almacena MBR en registro	**r1:=MBR**
0111	**TEST**	Examina registro	**N:=(r2<0); Z:=(r2==0)**
1000	**BEGRD**	Comienza la Lectura	**MAR:=r1; rd**
1001	**BEGWR**	Comienza la Escritura	**MAR:=r1; MBR:=r2; wr**
1010	**CONRD**	Continúa la Lectura	**rd**
1011	**CONWR**	Continúa la Escritura	**wr**
1100			
1101	**NJUMP**	Salta si N=true	**if N then goto r1<<4 | r2**
1110	**ZJUMP**	Salta si Z=true	**if Z then goto r1<<4 | r2**
1111	**UJUMP**	Salta siempre	**goto r1<<4 | r2**

Código de operación de microinstrucción		ALUL ALUH	SHL SHH	AMUX NZ	MAR AND	RD MBR	MSLH WR	MSLL
0	ADD			+	+			
1	AND		+	+	+			
2	MOVE	+		+	+			
3	COMPL	+	+	+	+			
4	LSHIFT	+	+		+			
5	RSHIFT	+	+		+			
6	GETMBR	+		+	+			
7	TEST	+		+				
8	BEGRD	+				+	+	
9	BEGWR	+				+	+	
10	CONRD	+					+	
11	CONWR	+					+	
12								
13	NJUMP	+						+
14	ZJUMP	+					+	
15	UJUMP	+					+	+



Binario	Mnemónico	Instrucción	Significado
0000	**ADD**	Suma	**r1:=r1+r2**
0001	**AND**	Y bit a bit	**r1:=r1&r2**
0010	**MOVE**	Mueve registro	**r1:=r2**
0011	**COMPL**	Complemento	**r1:=inv(r2)**

Código de operación de microinstrucción		ALUH	ALUL	SHH	SHL	NZ	AMUX	AND	MAR	MBR	RD	WR	MSLH	MSLL
0	ADD					+		+						
1	AND		+			+		+						
2	MOVE	+				+		+						
3	COMPL	+	+			+		+						
4	LSHIFT	+		+				+						
5	RSHIFT	+			+			+						
6	GETMBR	+					+	+						
7	TEST													
8														
9												+		
10	CONRD	+									+			
11	CONWR	+										+		
12														
13	NJUMP	+												+
14	ZJUMP	+											+	
15	UJUMP	+											+	+

NZ para conservar el resultado
AND para guardar el resultado

:=0)

2; wr

<4 | r2

<4 | r2

Binario	Mnemónico	Instrucción	Significado

Código de operación de microinstrucción		ALUH	ALUL	SHH	SHL	NZ	AMUX	AND	MAR	MBR	RD	WR	MSLH	MSLL
0	ADD					+		+						
1	AND		+			+		+						
2	MOVE	+				+		+						
3	COMPL	+	+			+		+						
4	LSHIFT	+		+		+		+						
5	RSHIFT	+			+	+		+						
6	GETMBR	+				+	+	+						
7	TEST	+												
8	BEGRD	+									+			
9	BEGWR	+								+		+		
10	CONRD	+									+			
11	CONWR	+										+		
12														
13	NJUMP	+												+
14	ZJUMP	+											+	
15	UJUMP	+											+	+

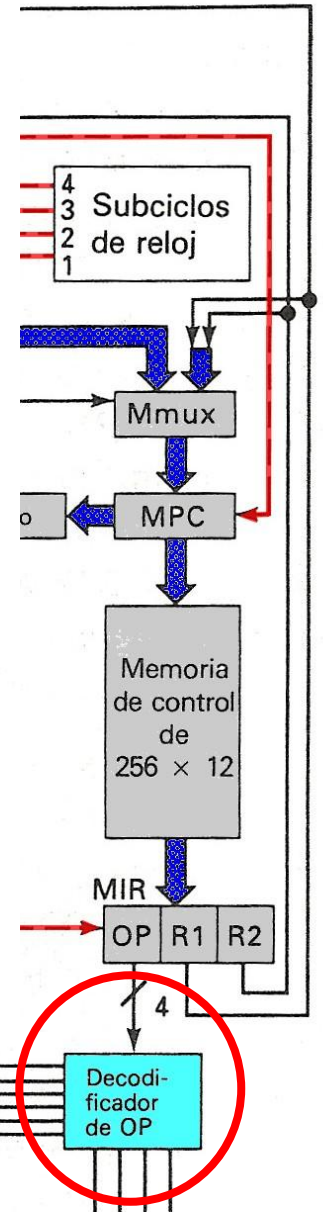
Deja pasar el dato sin modificaciones

:=0)

2; wr

1101	**NJUMP**	Salta si N=true	**if N then goto r1<<4 | r2**
1110	**ZJUMP**	Salta si Z=true	**if Z then goto r1<<4 | r2**
1111	**UJUMP**	Salta siempre	**goto r1<<4 | r2**

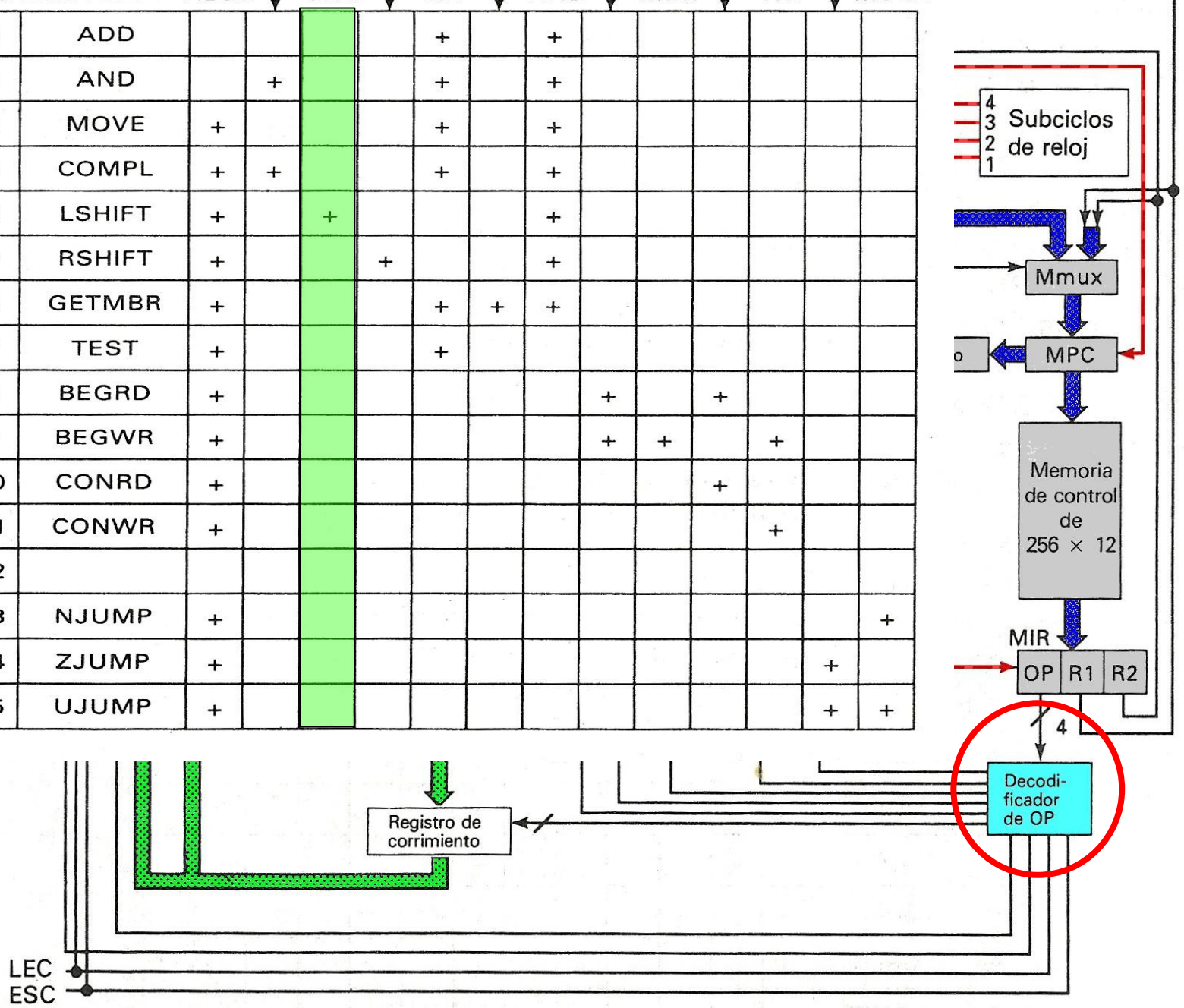
Código de operación de microinstrucción		ALUH	ALUL	SHH	SHL	NZ	AMUX	AND	MAR	MBR	RD	WR	MSLH	MSLL
0	ADD					+		+						
1	AND		+			+		+						
2	MOVE	+				+		+						
3	COMPL	+	+			+		+						
4	LSHIFT	+		+				+						
5	RSHIFT	+			+			+						
6	GETMBR	+				+	+	+						
7	TEST	+				+								
8	BEGRD	+							+		+			
9	BEGWR	+							+	+		+		
10	CONRD	+									+			
11	CONWR	+										+		
12														
13	NJUMP	+												+
14	ZJUMP												+	
15	U												+	+



¿Cómo se implementa el Decodificador de Operaciones?

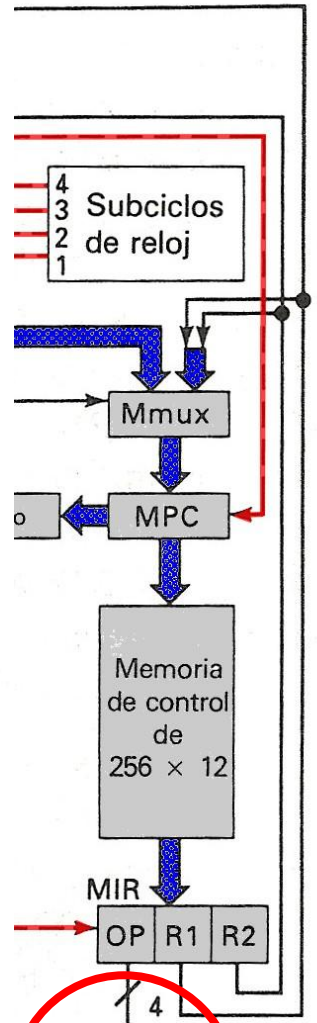
LEC
ESC

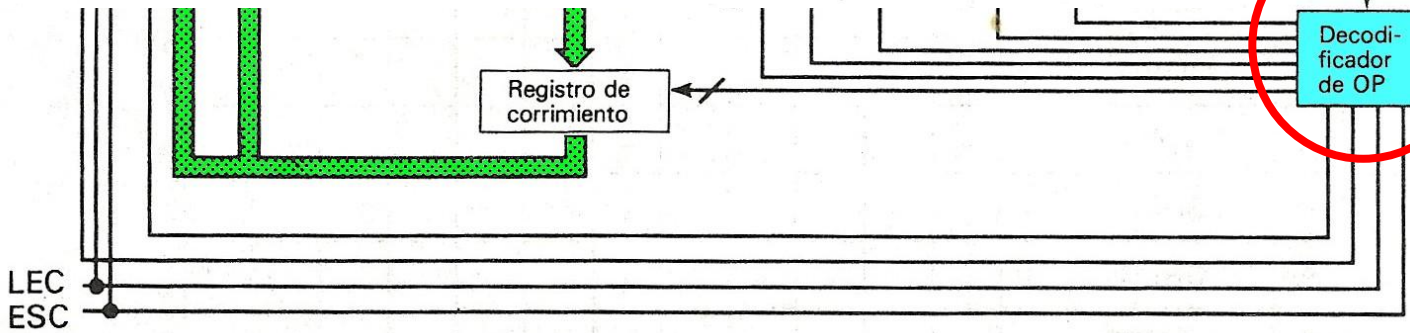
Código de operación de microinstrucción		ALUH	ALUL	SHH	SHL	NZ	AMUX	AND	MAR	MBR	RD	WR	MSLH	MSLL
0	ADD					+		+						
1	AND		+			+		+						
2	MOVE	+				+		+						
3	COMPL	+	+			+		+						
4	LSHIFT	+		+				+						
5	RSHIFT	+			+			+						
6	GETMBR	+				+	+	+						
7	TEST	+				+								
8	BEGRD	+							+		+			
9	BEGWR	+							+	+		+		
10	CONRD	+									+			
11	CONWR	+										+		
12														
13	NJUMP	+												+
14	ZJUMP	+											+	
15	UJUMP	+											+	+



Código de operación de microinstrucción		ALUH	ALUL	SHH	SHL	NZ	AMUX	AND	MAR	MBR	RD	WR	MSLH	MSLL
0	ADD					+		+						
1	AND		+			+		+						
2	MOVE	+				+		+						
3	COMPL	+	+			+		+						
4	LSHIFT	+		+				+						
5	RSHIFT	+			+									
6	GETMBR	+												
7	TEST	+												
8	BEGRD	+							+		+			
9	BEGWR	+							+	+		+		
10	CONRD	+									+			
11	CONWR	+										+		
12														
13	NJUMP	+												+
14	ZJUMP	+											+	
15	UJUMP	+											+	+

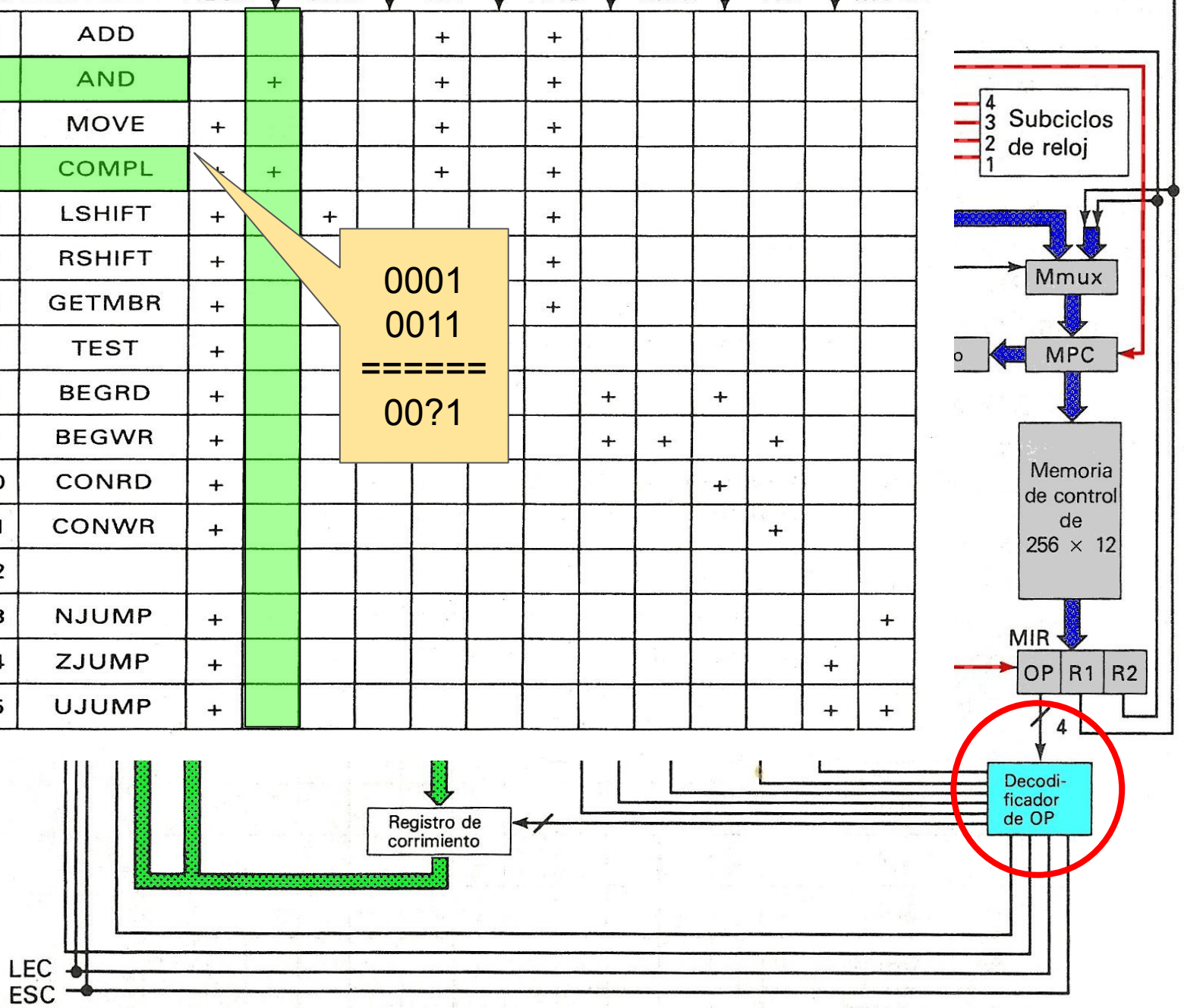
0100





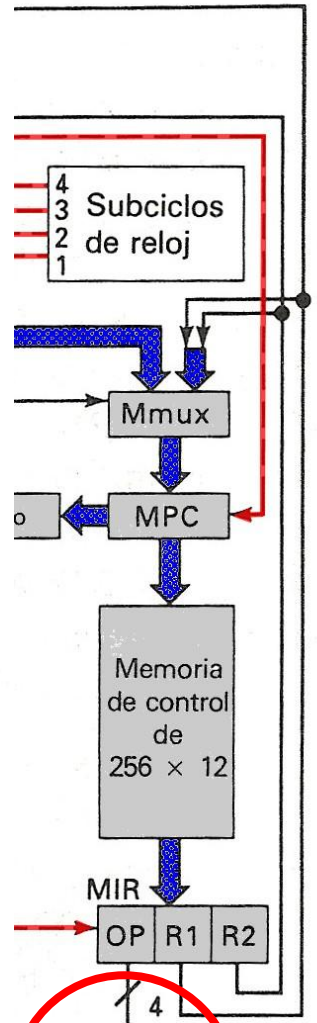
Código de operación de microinstrucción		ALUH	ALUL	SHH	SHL	NZ	AMUX	AND	MAR	MBR	RD	WR	MSLH	MSLL
0	ADD					+		+						
1	AND		+			+		+						
2	MOVE	+				+		+						
3	COMPL	+	+			+		+						
4	LSHIFT	+		+				+						
5	RSHIFT	+						+						
6	GETMBR	+						+						
7	TEST	+												
8	BEGRD	+							+		+			
9	BEGWR	+							+	+		+		
10	CONRD	+									+			
11	CONWR	+										+		
12														
13	NJUMP	+												+
14	ZJUMP	+											+	
15	UJUMP	+											+	+

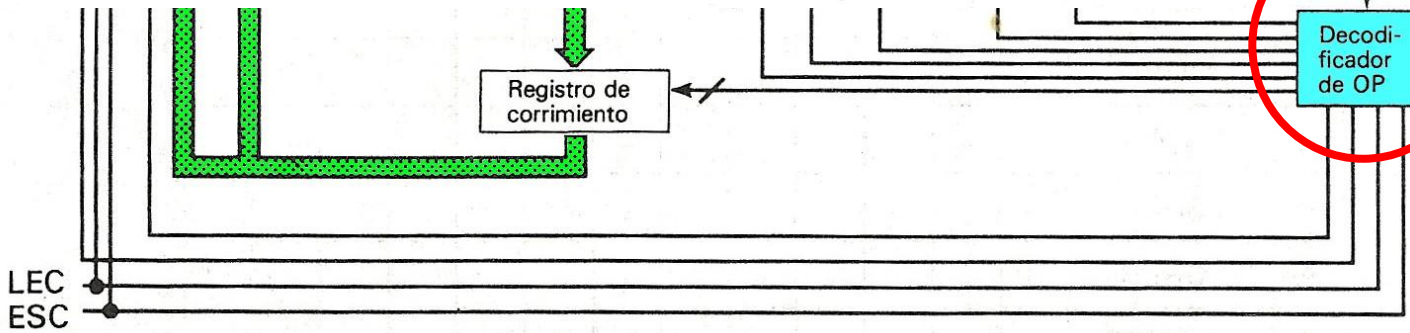
0001
0011
======
00?1



Código de operación de microinstrucción		ALUH	ALUL	SHH	SHL	NZ	AMUX	AND	MAR	MBR	RD	WR	MSLH	MSLL
0	ADD					+		+						
1	AND		+			+		+						
2	MOVE	+				+		+						
3	COMPL	+	+					+						
4	LSHIFT	+		+				+						
5	RSHIFT	+						+						
6	GETMBR	+						+						
7	TEST	+												
8	BEGRD	+							+		+			
9	BEGWR	+							+	+		+		
10	CONRD	+									+			
11	CONWR	+										+		
12														
13	NJUMP	+												+
14	ZJUMP	+											+	
15	UJUMP	+											+	+

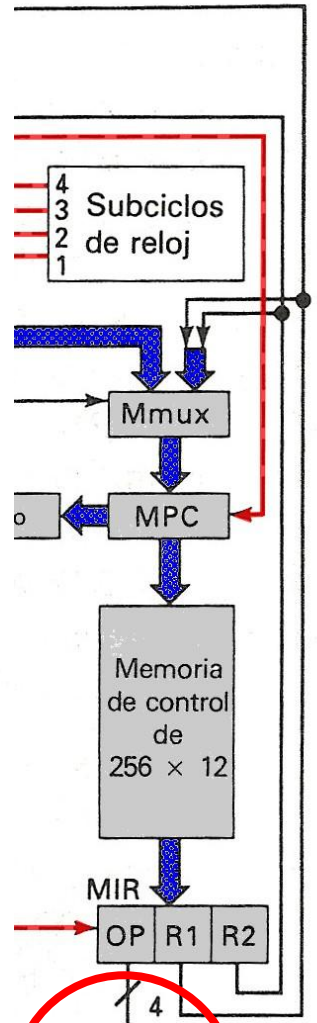
1000
1001
======
100?

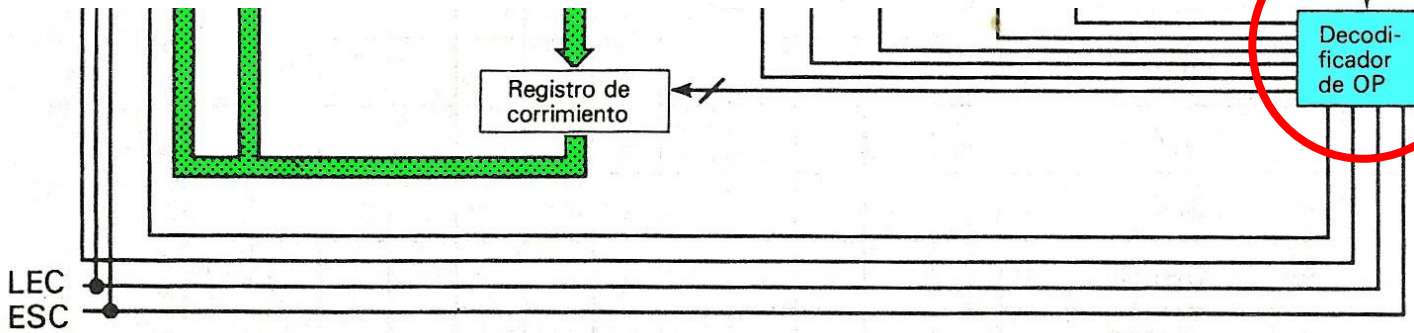




Código de operación de microinstrucción		ALUH	ALUL	SHH	SHL	NZ	AMUX	AND	MAR	MBR	RD	WR	MSLH	MSLL
0	ADD					+		+						
1	AND		+			+		+						
2	MOVE	+				+		+						
3	COMPL	+	+			+		+						
4	LSHIFT	+		+				+						
5	RSHIFT	+			+			+						
6	GETMBR	+				+	+	+						
7	TEST	+				+								
8	BEGRD	+							+		+			
9	BEGWR	+							+	+		+		
10	CONRD	+									+			
11	CONWR	+										+		
12														
13	NJUMP	+												+
14	ZJUMP	+											+	
15	UJUMP	+											+	+

1110
1111
======
111?





```
 0: mar := pc; rd;
 1: rd;
    pc := pc + 1;
 2: ir := mbr;
    tir := lshift(ir);
    if n then goto 28;
 3: tir := lshift(tir);
    if n then goto 19;
 4: tir := lshift(tir);
    if n then goto 11;
 5: alu := tir;
    if n then goto 09;
 6: mar := ir; rd;    {LODD}
 7: rd;
 8: ac := mbr;
    goto 0;
 9: mar := ir; mbr:= ac; wr;
                      {STOD}
10: wr
    goto 0;
11: alu := tir;
    if n then goto 15;
12: mar := ir; rd;    {ADDD}
13: rd;
14: a := mbr;
    ac := ac + a;
    goto 0;
15: mar := ir; rd;    {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
    a := inv(a);
18: ac := ac + a;
    goto 0;
19: tir := Ishift(tir);
    if n then goto 25;
20: alu := tir;
    if n then goto 23;
21: alu := ac;        {JPOS}
    if n then goto 0;
22: pc := ir;
    pc band(pc, amask);
    goto 0;
23: alu := ac;        {IZER}
    if z then goto 22;
24: goto 0;
25: alu := tir;
    if n then goto 27;
26: pc:= ir;          {JUMP}
    pc:= band(pc, amask);
    goto 0;
27: ac := ir;         {LOCO}
    ac :=band(ac, amask);
    goto 0;
28: tir := lshift (tir):
    if n then goto 40;
29: tir:= lshift (tir);
    if n then goto 35;
30: alu := tir;
    if n then goto 33;
31: a := ir;          {LODL}
    a := a + sp;
32: mar := a; rd;
    rd;
    ac := mbr;
    goto 0
33: a := ir;          {STOL}
    a := a + sp;
34: mar := a; mbr := ac, wr;
    wr;
    goto 0;
35: alu := tir;
    if n then goto 38;
36: a := ir;          {ADDL}
    a := a + sp;
37: mar := a; rd;
    rd;
    a := mbr;
    ac := ac + a;
    goto 0;
38: a := ir;          {SUBL}
    a := a + sp;
39: mar := a; rd;
    rd;
    goto 99;
40: tir := lshift (tir );
    if n then goto 46;
41: alu:= tir;
    if n then goto 44;
42: alu:= ac;         {JNEG}
    if n then goto 22;
43: goto 0;
44: alu := ac;        {JNZE}
    if z then goto 0;
45: pc := ir;
    pc := band (pc, amask);
    goto 0;
46: tir := lshift(tir);
    if n then goto 50;
47: sp := sp + (-1); {CALL}
48: mar := sp; mbr:= pc; wr;
    wr;
49: pc := ir;
    pc := band(pc, amask);
    goto 0;
50: tir := lshift (tir );
    if n then goto 55;
    tir := lshift
    then go
52: a
54:
    mar :=
    wr;
    goto
56: mar := sp; rd      {POPI}
57: rd;
    sp := sp + 1;
58: a := mbr;
    mar := ac; mbr := a; wr;
    wr;
    goto 0;
59: alu := tir;
    if n then goto 62;
60: sp := sp + (-1); {PUSH}
61: mar:= sp; mbr :=ac; wr;
    wr;
    goto 0;
62: mar := sp; rd;    {POP}
63: rd;
    sp := sp + 1;
64: ac := mbr;
    goto 0;
65: tir := lshift (tir ):
    if n then goto 73;
66: alu := tir;
    if n then goto 70;
67: mar := sp; rd;    {RETN}
68: rd;
    sp := sp + 1;
69: pc := mbr;
    goto 0;
70: a := ac;          {SWAP}
    ac := sp;
    sp:= a;
    goto 0;
    en goto 76;
    {INSP}
    smask);
    a;
    {DESP}
    d(a, smask);
    a);
    a := a
    sp := sp + a;
    goto 0;
```

Micro-Programa Vertical

```
 0: mar := pc; rd;
 1: rd;
    pc := pc + 1;
 2: ir := mbr;
    tir := lshift(ir);
    if n then goto 28;
 3: tir := lshift(tir);
    if n then goto 19;
 4: tir := lshift(tir);
    if n then goto
 5: alu := tir;
    if n then goto
 6: mar := ir;
 7: rd;
 8:
 9:                    {STOD}
10: wr
    got 0;
11: alu := tir;
    if n then goto 15;
12: mar := ir; rd;    {ADDD}
13: rd;
14: a := mbr;
    ac := ac + a;
    goto 0;
15: mar := ir; rd;    {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
    a := inv(a);
18: ac := ac + a;
    goto 0;
19: tir := Ishift(tir);
    if n then goto 25;
20: alu := tir;
    if n then goto 23;
21: alu := ac;        {JPOS}
    if n then goto 0;
22: pc := ir;
    pc band(pc, amask);
    goto 0;
23: alu := ac;        {IZER}
```

Micro-Programa Horizontal

```
0: mar := pc ; rd ;                          {ciclo principal}
1: pc := pc + 1; rd ;                        {incrementa pe}
2: ir := mbr ; if n then goto 28;            {salva y decodifica mbr}
3: tir := lshift (ir + ir ); if n then goto 19;
4: tir := lshift (tir ); if n then goto 11;  {000x o 001x?}
5: alu := tir ; if n then goto 9;            {0000 o 0001?}
6: mar := ir ; rd ;                          {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0;
```

```
29: tir:= lshift (tir);
    if n then goto 35;
30: alu := tir;
    if n then goto 33;
31: a := ir;          {LODL}
    a := a + sp;
32: mar := a; rd;
    rd;
    ac := mbr;
    goto 0
33: a := ir;          {STOL}
    a := a + sp;
34: mar := a; mbr := ac, wr;
    wr;
    goto 0;
35: alu := tir;
    if n then goto 38;
36: a := ir;          {ADDL}
    a := a + sp;
37: mar := a; rd;
    rd;
    a := mbr;
    ac := ac + a;
    goto 0;
38: a := ir;          {SUBL}
```

```
    if z then goto 0;
45: pc := ir;
    pc := band (pc, amask);
    goto 0;
46: tir := lshift(tir);
    if n then goto 50;
47: sp := sp + (-1); {CALL}
48: mar := sp; mbr:= pc; wr;
    wr;
49: pc := ir;
    pc := band(pc, amask);
    goto 0;
50: tir := lshift (tir );
    if n then got 55;
    tir := lshift
    then go
52: a
54:
    mar :=
    wr;
    goto
56: mar := sp; rd      {POPI}
57: rd;
    sp := sp + 1;
```

Micro-Programa Vertical

```
58: a := mbr;
    mar := ac; mbr := a; wr;
    wr;
    goto 0;
    alu := tir;
    if n then goto 62;
    sp := sp + (-1); {PUSH}
    mar:= sp; mbr :=ac; wr;
    wr;
    goto 0;
    mar := sp; rd;    {POP}
    rd;
    sp := sp + 1;
64: ac := mbr;
    goto 0;
65: tir := lshift (tir ):
    if n then goto 73;
66: alu := tir;
    if n then goto 70;
67: mar := sp; rd;    {RETN}
68: rd;
    sp := sp + 1;
69: pc := mbr;
    goto 0;
70: a := ac;          {SWAP}
    ac := sp;
    sp:= a;
    goto 0;
    en goto 76;
    {INSP}
    smask);
    a;
    {DESP}
    d(a, smask);
    a);
    a := a
    sp := sp + a;
    goto 0;
```

```
 0: mar := pc; rd;
 1: rd;
    pc := pc + 1;
 2: ir := mbr;
    tir := lshift(ir);
    if n then goto 28;
 3: tir := lshift(tir);
    if n then goto 19;
 4: tir := lshift(tir);
    if n then goto 11;
 5: alu := tir;
    if n then goto 09;
 6: mar := ir; rd;    {LODD}
 7: rd;
 8: ac := mbr;
    goto 0;
 9: mar := ir; mbr:= ac; wr;
                      {STOD}
10: wr
    goto 0;
11: alu := tir;
    if n then goto 15;
12: mar := ir; rd;    {ADDD}
13: rd;
14: a := mbr;
    ac := ac + a;
    goto 0;
15: mar := ir; rd;    {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
    a := inv(a);
18: ac := ac + a;
    goto 0;
19: tir := Ishift(tir);
    if n then goto 25;
20: alu := tir;
    if n then goto 23;
21: alu := ac;        {JPOS}
    if n then goto 0;
22: pc := ir;
    pc band(pc, amask);
    goto 0;
```

```
23: alu := ac;        {IZER}
```

0: *mar* := *pc* ; *rd* ;	{ciclo principal}
1: *pc* := *pc* + 1; *rd* ;	{incrementa pe}
2: *ir* := *mbr* ; **if** *n* **then goto** 28;	{salva y decodifica mbr}
3: *tir* := *lshift* (*ir* + *ir*); **if** *n* **then goto** 19;	
4: *tir* := *lshift* (*tir*); **if** *n* **then goto** 11;	{000x o 001x?}
5: *alu* := *tir* ; **if** *n* **then goto** 9;	{0000 o 0001?}
6: *mar* := *ir* ; *rd* ;	{0000 = LODD}
7: *rd* ;	
8: *ac* := *mbr* ; **goto** 0;	

```
29: tir:= lshift (tir);
    if n then goto 35;
30: alu := tir;
    if n then goto 33;
31: a := ir;          {LODL}
    a := a + sp;
32: mar := a; rd;
    rd;
    ac := mbr;
    goto 0
33: a := ir;          {STOL}
    a := a + sp;
34: mar := a; mbr := ac, wr;
    wr;
    goto 0;
35: alu := tir;
    if n then goto 38;
36: a := ir;          {ADDL}
    a := a + sp;
37: mar := a; rd;
    rd;
    a := mbr;
    ac := ac + a;
    goto 0;
```

```
38: a := ir;          {SUBL}
```

```
    if z then goto 0;
45: pc := ir;
    pc := band (pc, amask);
    goto 0;
46: tir := lshift(tir);
    if n then goto 50;
47: sp := sp + (-1); {CALL}
48: mar := sp; mbr:= pc; wr;
    wr;
49: pc := ir;
    pc := band(pc, amask);
    goto 0;
50: tir := lshift (tir );
    if n then got[illegible]5;
    [illegible] := lshift[illegible]
    [illegible] go[illegible]
52: a[illegible]
54:[illegible]
    mar :=[illegible]
    wr;
    goto[illegible]
56: mar := sp; rd[illegible]  {POPI}
57: rd;
    sp := sp + 1;
```

```
58: a := mbr;
    mar := ac; mbr := a; wr;
    wr;
    goto 0;
    alu := tir;
    if n then goto 62;
    sp := sp + (-1); {PUSH}
    mar:= sp; mbr :=ac; wr;
    wr;
    goto 0;
    mar := sp; rd;    {POP}
    rd;
    sp := sp + 1;
64: ac := mbr;
    goto 0;
65: tir := lshift (tir ):
    if n then goto 73;
66: alu := tir;
    if n then goto 70;
67: mar := sp; rd;    {RETN}
68: rd;
    sp := sp + 1;
69: pc := mbr;
    goto 0;
70: [illegible]a := ac;          {SWAP}
7[illegible] ac := sp;
    sp:= a;
    goto 0;
    [illegible]
    [illegible]en goto 76;
    [illegible]NSP}
    [illegible]mask);
    [illegible]a;
    [illegible]SP}
    [illegible]d(a, smask);
    [illegible] a :[illegible]a);
    [illegible]: a := a [illegible]
    sp := sp + a;
    goto 0;
```

Micro-Programa Vertical

```
0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);
   if n then goto 11;
5: alu := tir;
   if n then goto 09;
6: mar := ir; rd;    {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;
```

```
23: alu := ac;        {IZER}
```

0: *mar* := *pc* ; *rd* ;	{ciclo principal}
1: *pc* := *pc* + 1; *rd* ;	{incrementa pe}
2: *ir* := *mbr* ; **if** *n* **then goto** 28;	{salva y decodifica mbr}
3: *tir* := *lshift* (*ir* + *ir*); **if** *n* **then goto** 19;	
4: *tir* := *lshift* (*tir*); **if** *n* **then goto** 11;	{000x o 001x?}
5: *alu* := *tir* ; **if** *n* **then goto** 9;	{0000 o 0001?}
6: *mar* := *ir* ; *rd* ;	{0000 = LODD}
7: *rd* ;	
8: *ac* := *mbr* ; **goto** 0;	

```
29: tir:= lshift (tir);
    if n then goto 35;
30: alu := tir;
    if n then goto 33;
```

```
38: a := ir;          {SUBL}
    if z then goto 0;
45: pc := ir;
    pc := band (pc, amask);
    goto 0;
```

```
58: a := mbr;
    mar := ac; mbr := a; wr;
    wr;
    goto 0;
    alu := tir;
    if n then goto 62;
    sp := sp + (-1); {PUSH}
    mar:= sp; mbr :=ac; wr;
    wr;
    goto 0;
    mar := sp; rd;    {POP}
    rd;
    sp := sp + 1;
64: ac := mbr;
    goto 0;
65: tir := lshift (tir ):
    if n then goto 73;
```

1000	BEGRD	Comienza la Lectura	MAR:=r1; rd

```
11: alu := tir;
    if n then goto 15;
12: mar := ir; rd;    {ADDD}
13: rd;
14: a := mbr;
    ac := ac + a;
    goto 0;
15: mar := ir; rd;    {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
    a := inv(a);
18: ac := ac + a;
    goto 0;
19: tir := Ishift(tir);
    if n then goto 25;
20: alu := tir;
    if n then goto 23;
21: alu := ac;        {JPOS}
    if n then goto 0;
22: pc := ir;
    pc band(pc, amask);
    goto 0;
```

```
    rd;
    ac := mbr;
    goto 0
33: a := ir;          {STOL}
    a := a + sp;
34: mar := a; mbr := ac, wr;
    wr;
    goto 0;
35: alu := tir;
    if n then goto 38;
36: a := ir;          {ADDL}
    a := a + sp;
37: mar := a; rd;
    rd;
    a := mbr;
    ac := ac + a;
    goto 0;
```

```
48: mar := sp; mbr:= pc; wr;
    wr;
49: pc := ir;
    pc := band(pc, amask);
    goto 0;
50: tir := lshift (tir );
    if n then goto 55;
    tir := lshift
    then go
52: a
54:
    mar :=
    wr;
    goto
56: mar := sp; rd    {POPI}
57: rd;
    sp := sp + 1;
```

```
68: rd;
    sp := sp + 1;
69: pc := mbr;
    goto 0;
70: a := ac;          {SWAP}
    ac := sp;
    sp:= a;
    goto 0;
    en goto 76;
    {INSP}
    smask);
    a;
    {DESP}
    d(a, smask);
    a);
    a := a
    sp := sp + a;
    goto 0;
```

Micro-Programa Vertical

```
 0: mar := pc; rd;
 1: rd;
    pc := pc + 1;
 2: ir := mbr;
    tir := lshift(ir);
    if n then goto 28;
 3: tir := lshift(tir);
    if n then goto 19;
 4: tir := lshift(tir);
    if n then goto 11;
 5: alu := tir;
    if n then goto 09;
 6: mar := ir; rd;    {LODD}
 7: rd;
 8: ac := mbr;
    goto 0;
 9: mar := ir; mbr:= ac; wr;
                      {STOD}
10: wr
    goto 0;
11: alu := tir;
    if n then goto 15;
12: mar := ir; rd;    {ADDD}
13: rd;
14: a := mbr;
    ac := ac + a;
    goto 0;
15: mar := ir; rd;    {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
    a := inv(a);
18: ac := ac + a;
    goto 0;
19: tir := Ishift(tir);
    if n then goto 25;
20: alu := tir;
    if n then goto 23;
21: alu := ac;        {JPOS}
    if n then goto 0;
22: pc := ir;
    pc band(pc, amask);
    goto 0;
```

```
23: alu := ac;        {IZER}
```

0: *mar* := *pc* ; *rd* ;	{ciclo principal}
1: *pc* := *pc* + 1; *rd* ;	{incrementa pe}
2: *ir* := *mbr* ; **if** *n* **then goto** 28;	{salva y decodifica mbr}
3: *tir* := *lshift* (*ir* + *ir*); **if** *n* **then goto** 19;	
4: *tir* := *lshift* (*tir*); **if** *n* **then goto** 11;	{000x o 001x?}
5: *alu* := *tir* ; **if** *n* **then goto** 9;	{0000 o 0001?}
6: *mar* := *ir* ; *rd* ;	{0000 = LODD}
7: *rd* ;	
8: *ac* := *mbr* ; **goto** 0;	

```
29: tir:= lshift (tir);
    if n then goto 35;
30: alu := tir;
    if n then goto 33;
31: a := ir;          {LODL}
    a := a + sp;
32: mar := a; rd;
    rd;
    ac := mbr;
    goto 0
33: a := ir;          {STOL}
    a := a + sp;
34: mar := a; mbr := ac, wr;
    wr;
    goto 0;
35: alu := tir;
    if n then goto 38;
36: a := ir;          {ADDL}
    a := a + sp;
37: mar := a; rd;
    rd;
    a := mbr;
    ac := ac + a;
    goto 0;
```

```
38: a := ir;          {SUBL}
```

```
    if z then goto 0;
45: pc := ir;
    pc := band (pc, amask);
    goto 0;
46: tir := lshift(tir);
    if n then goto 50;
47: sp := sp + (-1); {CALL}
48: mar := sp; mbr:= pc; wr;
    wr;
49: pc := ir;
    pc := band(pc, amask);
    goto 0;
50: tir := lshift (tir );
    if n then got[illegible]5;
    [illegible] := lshift[illegible]
    [illegible] go[illegible]
52: a[illegible]
54:[illegible]
    mar :=[illegible]
    wr;
    goto[illegible]
56: mar := sp; rd[illegible]{POPI}
57: rd;
    sp := sp + 1;
```

```
58: a := mbr;
    mar := ac; mbr := a; wr;
    wr;
    goto 0;
    alu := tir;
    if n then goto 62;
    sp := sp + (-1); {PUSH}
    mar:= sp; mbr :=ac; wr;
    wr;
    goto 0;
    mar := sp; rd;    {POP}
    rd;
    sp := sp + 1;
64: ac := mbr;
    goto 0;
65: tir := lshift (tir ):
    if n then goto 73;
66: alu := tir;
    if n then goto 70;
67: mar := sp; rd;    {RETN}
68: rd;
    sp := sp + 1;
69: pc := mbr;
    goto 0;
70: [illegible]a := ac;          {SWAP}
7[illegible] ac := sp;
    sp:= a;
    goto 0;
    [illegible]
    [illegible]en goto 76;
    [illegible]{INSP}
    [illegible]smask);
    [illegible]a;
    [illegible]{DESP}
    [illegible]d(a, smask);
    [illegible] a :[illegible]a);
    [illegible]: a := a [illegible]
    sp := sp + a;
    goto 0;
```

Micro-Programa Vertical

```
0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);
   if n then goto 11;
5: alu := tir;
   if n then goto 09;
6: mar := ir; rd;    {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;
```

```
0: mar := pc ; rd ;                          {ciclo principal}
1: pc := pc + 1; rd ;                        {incrementa pc}
2: ir := mbr ; if n then goto 28;            {salva y decodifica mbr}
3: tir := lshift (ir + ir ); if n then goto 19;
4: tir := lshift (tir ); if n then goto 11;  {000x o 001x?}
5: alu := tir ; if n then goto 9;            {0000 o 0001?}
6: mar := ir ; rd ;                          {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0;
```

```
23: alu := ac;        {IZER}
29: tir:= lshift (tir);
    if n then goto 35;
30: alu := tir;
    if n then goto 33;
```

```
38: a := ir;          {SUBL}
    if z then goto 0;
45: pc := ir;
    pc := band (pc, amask);
    goto 0;
```

```
58: a := mbr;
    mar := ac; mbr := a; wr;
    wr;
    goto 0;
    alu := tir;
    if n then goto 62;
    sp := sp + (-1); {PUSH}
    mar:= sp; mbr :=ac; wr;
    wr;
    goto 0;
    mar := sp; rd;    {POP}
    rd;
    sp := sp + 1;
64: ac := mbr;
    goto 0;
65: tir := lshift (tir ):
    if n then goto 73;
```

1010	CONRD	Continúa la Lectura	rd

```
11: alu := tir;
    if n then goto 15;
12: mar := ir; rd;    {ADDD}
13: rd;
14: a := mbr;
    ac := ac + a;
    goto 0;
15: mar := ir; rd;    {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
    a := inv(a);
18: ac := ac + a;
    goto 0;
19: tir := Ishift(tir);
    if n then goto 25;
20: alu := tir;
    if n then goto 23;
21: alu := ac;        {JPOS}
    if n then goto 0;
22: pc := ir;
    pc band(pc, amask);
    goto 0;
```

```
    rd;
    ac := mbr;
    goto 0
33: a := ir;          {STOL}
    a := a + sp;
34: mar := a; mbr := ac, wr;
    wr;
    goto 0;
35: alu := tir;
    if n then goto 38;
36: a := ir;          {ADDL}
    a := a + sp;
37: mar := a; rd;
    rd;
    a := mbr;
    ac := ac + a;
    goto 0;
```

```
48: mar := sp; mbr:= pc; wr;
    wr;
49: pc := ir;
    pc := band(pc, amask);
    goto 0;
50: tir := lshift (tir );
    if n then goto 55;
    tir := lshift
    then go
52: a
54:
    mar :=
    wr;
    goto
56: mar := sp; rd    {POPI}
57: rd;
    sp := sp + 1;
```

```
68: rd;
    sp := sp + 1;
69: pc := mbr;
    goto 0;
70: a := ac;          {SWAP}
    ac := sp;
    sp:= a;
    goto 0;
    en goto 76;
    {INSP}
    smask);
    a;
    {DESP}
    d(a, smask);
    a);
    a := a
    sp := sp + a;
    goto 0;
```

Micro-Programa Vertical

```
0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);
   if n then goto 11;
5: alu := tir;
   if n then goto 09;
6: mar := ir; rd;    {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;
```

```
23: alu := ac;        {IZER}
```

```
38: a := ir;          {SUBL}
```

```
58: a := mbr;
    mar := ac; mbr := a; wr;
    wr;
    goto 0;
    alu := tir;
    if n then goto 62;
    sp := sp + (-1); {PUSH}
    mar:= sp; mbr :=ac; wr;
    wr;
    goto 0;
    mar := sp; rd;    {POP}
    rd;
    sp := sp + 1;
```

```
0: mar := pc ; rd ;                    {ciclo principal}
1: pc := pc + 1; rd ;                  {incrementa pe}
2: ir := mbr ; if n then goto 28;      {salva y decodifica mbr}
3: tir := lshift (ir + ir ); if n then goto 19;
4: tir := lshift (tir ); if n then goto 11;   {000x o 001x?}
5: alu := tir ; if n then goto 9;             {0000 o 0001?}
6: mar := ir ; rd ;                    {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0;
```

```
29: tir:= lshift (tir);
    if n then goto 35;
30: alu := tir;
    if n then goto 33;
```

```
    if z then goto 0;
45: pc := ir;
    pc := band (pc, amask);
    goto 0;
```

```
64: ac := mbr;
    goto 0;
65: tir := lshift (tir ):
    if n then goto 73;
```

1010	**CONRD**	Continúa la Lectura	**rd**
0000	**ADD**	Suma	**r1:=r1+r2**

```
12: mar := ir; rd;    {ADDD}
13: rd;
14: a := mbr;
    ac := ac + a;
    goto 0;
15: mar := ir; rd;    {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
    a := inv(a);
18: ac := ac + a;
    goto 0;
19: tir := Ishift(tir);
    if n then goto 25;
20: alu := tir;
    if n then goto 23;
21: alu := ac;        {JPOS}
    if n then goto 0;
22: pc := ir;
    pc band(pc, amask);
    goto 0;
```

```
    goto 0
33: a := ir;          {STOL}
    a := a + sp;
34: mar := a; mbr := ac, wr;
    wr;
    goto 0;
35: alu := tir;
    if n then goto 38;
36: a := ir;          {ADDL}
    a := a + sp;
37: mar := a; rd;
    rd;
    a := mbr;
    ac := ac + a;
    goto 0;
```

```
49: pc := ir;
    pc := band(pc, amask);
    goto 0;
50: tir := lshift (tir );
    if n then goto 55;
    tir := lshift
    then go
52: a
54:
    mar :=
    wr;
    goto
56: mar := sp; rd    {POPI}
57: rd;
    sp := sp + 1;
```

```
69: pc := mbr;
    goto 0;
70: a := ac;          {SWAP}
    ac := sp;
    sp:= a;
    goto 0;
    ;
    en goto 76;
    {INSP}
    smask);
    a;
    {DESP}
    d(a, smask);
    a);
    a := a
    sp := sp + a;
    goto 0;
```

Micro-Programa Vertical

```
 0: mar := pc; rd;
 1: rd;
    pc := pc + 1;
 2: ir := mbr;
    tir := lshift(ir);
    if n then goto 28;
 3: tir := lshift(tir);
    if n then goto 19;
 4: tir := lshift(tir);
    if n then goto 11;
 5: alu := tir;
    if n then goto 09;
 6: mar := ir; rd;    {LODD}
 7: rd;
 8: ac := mbr;
    goto 0;
 9: mar := ir; mbr:= ac; wr;
                      {STOD}
10: wr
    goto 0;
11: alu := tir;
    if n then goto 15;
12: mar := ir; rd;    {ADDD}
13: rd;
14: a := mbr;
    ac := ac + a;
    goto 0;
15: mar := ir; rd;    {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
    a := inv(a);
18: ac := ac + a;
    goto 0;
19: tir := Ishift(tir);
    if n then goto 25;
20: alu := tir;
    if n then goto 23;
21: alu := ac;        {JPOS}
    if n then goto 0;
22: pc := ir;
    pc band(pc, amask);
    goto 0;
```

```
23: alu := ac;        {IZER}
```

0: *mar* := *pc* ; *rd* ;	{ciclo principal}
1: *pc* := *pc* + 1; *rd* ;	{incrementa pc}
2: *ir* := *mbr* ; **if** *n* **then goto** 28;	{salva y decodifica mbr}
3: *tir* := *lshift* (*ir* + *ir*); **if** *n* **then goto** 19;	
4: *tir* := *lshift* (*tir*); **if** *n* **then goto** 11;	{000x o 001x?}
5: *alu* := *tir* ; **if** *n* **then goto** 9;	{0000 o 0001?}
6: *mar* := *ir* ; *rd* ;	{0000 = LODD}
7: *rd* ;	
8: *ac* := *mbr* ; **goto** 0;	

```
29: tir:= lshift (tir);
    if n then goto 35;
30: alu := tir;
    if n then goto 33;
31: a := ir;          {LODL}
    a := a + sp;
32: mar := a; rd;
    rd;
    ac := mbr;
    goto 0
33: a := ir;          {STOL}
    a := a + sp;
34: mar := a; mbr := ac, wr;
    wr;
    goto 0;
35: alu := tir;
    if n then goto 38;
36: a := ir;          {ADDL}
    a := a + sp;
37: mar := a; rd;
    rd;
    a := mbr;
    ac := ac + a;
    goto 0;
```

```
38: a := ir;          {SUBL}
```

```
    if z then goto 0;
45: pc := ir;
    pc := band (pc, amask);
    goto 0;
46: tir := lshift(tir);
    if n then goto 50;
47: sp := sp + (-1); {CALL}
48: mar := sp; mbr:= pc; wr;
    wr;
49: pc := ir;
    pc := band(pc, amask);
    goto 0;
50: tir := lshift (tir );
    if n then got[illegible]5;
    [illegible] := lshift[illegible]
    [illegible] gc[illegible]
52: a[illegible]
54:[illegible]
    mar :=[illegible]
    wr;
    goto[illegible]
56: mar := sp; rd[illegible]  {POPI}
57: rd;
    sp := sp + 1;
```

```
58: a := mbr;
    mar := ac; mbr := a; wr;
    wr;
    goto 0;
    alu := tir;
    if n then goto 62;
    sp := sp + (-1); {PUSH}
    mar:= sp; mbr :=ac; wr;
    wr;
    goto 0;
    mar := sp; rd;    {POP}
    rd;
    sp := sp + 1;
64: ac := mbr;
    goto 0;
65: tir := lshift (tir ):
    if n then goto 73;
66: alu := tir;
    if n then goto 70;
67: mar := sp; rd;    {RETN}
68: rd;
    sp := sp + 1;
69: pc := mbr;
    goto 0;
70: [illegible]a := ac;          {SWAP}
7[illegible] ac := sp;
    sp:= a;
    goto 0;
    [illegible]
    [illegible]en goto 76;
    [illegible]NSP}
    [illegible]mask);
    [illegible]a;
    [illegible]SP}
    [illegible]d(a, smask);
    [illegible] a [illegible]a);
    [illegible]: a := a [illegible]
    sp := sp + a;
    goto 0;
```

Micro-Programa Vertical

```
0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);
   if n then goto 11;
5: alu := tir;
   if n then goto 09;
6: mar := ir; rd;    {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;
```

```
23: alu := ac;        {IZER}
```

```
38: a := ir;          {SUBL}
```

```
58: a := mbr;
    mar := ac; mbr := a; wr;
    wr;
    goto 0;
    alu := tir;
    if n then goto 62;
    sp := sp + (-1); {PUSH}
    mar:= sp; mbr :=ac; wr;
    wr;
    goto 0;
    mar := sp; rd;    {POP}
    rd;
    sp := sp + 1;
```

0: *mar* := *pc* ; *rd* ;	{ciclo principal}
1: *pc* := *pc* + 1; *rd* ;	{incrementa pe}
2: *ir* := *mbr* ; **if** *n* **then goto** 28;	{salva y decodifica mbr}
3: *tir* := *lshift* (*ir* + *ir*); **if** *n* **then goto** 19;	
4: *tir* := *lshift* (*tir*); **if** *n* **then goto** 11;	{000x o 001x?}
5: *alu* := *tir* ; **if** *n* **then goto** 9;	{0000 o 0001?}
6: *mar* := *ir* ; *rd* ;	{0000 = LODD}
7: *rd* ;	
8: *ac* := *mbr* ; **goto** 0;	

```
29: tir:= lshift (tir);
    if n then goto 35;
30: alu := tir;
    if n then goto 33;
```

```
    if z then goto 0;
45: pc := ir;
    pc := band (pc, amask);
    goto 0;
```

```
64: ac := mbr;
    goto 0;
65: tir := lshift (tir ):
    if n then goto 73;
```

0110	**GETMBR**	Almacena MBR en registro	**r1:=MBR**

```
11: alu := tir;
    if n then goto 15;
12: mar := ir; rd;    {ADDD}
13: rd;
14: a := mbr;
    ac := ac + a;
    goto 0;
15: mar := ir; rd;    {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
    a := inv(a);
18: ac := ac + a;
    goto 0;
19: tir := Ishift(tir);
    if n then goto 25;
20: alu := tir;
    if n then goto 23;
21: alu := ac;        {JPOS}
    if n then goto 0;
22: pc := ir;
    pc band(pc, amask);
    goto 0;
```

```
    rd;
    ac := mbr;
    goto 0
33: a := ir;          {STOL}
    a := a + sp;
34: mar := a; mbr := ac, wr;
    wr;
    goto 0;
35: alu := tir;
    if n then goto 38;
36: a := ir;          {ADDL}
    a := a + sp;
37: mar := a; rd;
    rd;
    a := mbr;
    ac := ac + a;
    goto 0;
```

```
48: mar := sp; mbr:= pc; wr;
    wr;
49: pc := ir;
    pc := band(pc, amask);
    goto 0;
50: tir := lshift (tir );
    if n then goto 55;
    tir := lshift
    then go
52: a
54:
    mar :=
    wr;
    goto
56: mar := sp; rd    {POPI}
57: rd;
    sp := sp + 1;
```

```
68: rd;
    sp := sp + 1;
69: pc := mbr;
    goto 0;
70: a := ac;          {SWAP}
    ac := sp;
    sp:= a;
    goto 0;
    en goto 76;
    {INSP}
    smask);
    a;
    {DESP}
    d(a, smask);
    a);
    a := a
    sp := sp + a;
    goto 0;
```

Micro-Programa Vertical

```
0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);
   if n then goto 11;
5: alu := tir;
   if n then goto 09;
6: mar := ir; rd;    {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;
```

```
23: alu := ac;        {IZER}
```

```
38: a := ir;          {SUBL}
```

```
58: a := mbr;
    mar := ac; mbr := a; wr;
    wr;
    goto 0;
    alu := tir;
    if n then goto 62;
    sp := sp + (-1); {PUSH}
    mar:= sp; mbr :=ac; wr;
    wr;
    goto 0;
    mar := sp; rd;    {POP}
    rd;
    sp := sp + 1;
```

0: *mar* := *pc* ; *rd* ;			{ciclo principal}
1: *pc* := *pc* + 1; *rd* ;			{incrementa pc}
2: *ir* := *mbr* ; **if** *n* **then goto** 28;			{salva y decodifica mbr}
3: *tir* := *lshift* (*ir* + *ir*); **if** *n* **then goto** 19;			
4: *tir* := *lshift* (*tir*); **if** *n* **then goto** 11;			{000x o 001x?}
5: *alu* := *tir* ; **if** *n* **then goto** 9;			{0000 o 0001?}
6: *mar* := *ir* ; *rd* ;			{0000 = LODD}
7: *rd* ;			
8: *ac* := *mbr* ; **goto** 0;			

```
29: tir:= lshift (tir);
    if n then goto 35;
30: alu := tir;
    if n then goto 33;
```

```
    if z then goto 0;
45: pc := ir;
    pc := band (pc, amask);
    goto 0;
```

```
64: ac := mbr;
    goto 0;
65: tir := lshift (tir ):
    if n then goto 73;
```

0110	**GETMBR**	Almacena MBR en registro	**r1:=MBR**
0100	**LSHIFT**	Desplaza a la Izquierda	**r1:=r2<<1**

```
12: mar := ir; rd;    {ADDD}
13: rd;
14: a := mbr;
    ac := ac + a;
    goto 0;
15: mar := ir; rd;    {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
    a := inv(a);
18: ac := ac + a;
    goto 0;
19: tir := Ishift(tir);
    if n then goto 25;
20: alu := tir;
    if n then goto 23;
21: alu := ac;        {JPOS}
    if n then goto 0;
22: pc := ir;
    pc band(pc, amask);
    goto 0;
```

```
    goto 0
33: a := ir;          {STOL}
    a := a + sp;
34: mar := a; mbr := ac, wr;
    wr;
    goto 0;
35: alu := tir;
    if n then goto 38;
36: a := ir;          {ADDL}
    a := a + sp;
37: mar := a; rd;
    rd;
    a := mbr;
    ac := ac + a;
    goto 0;
```

```
49: pc := ir;
    pc := band(pc, amask);
    goto 0;
50: tir := lshift (tir );
    if n then goto 55;
    tir := lshift
    then go
52: a
54:
    mar :=
    wr;
    goto
56: mar := sp; rd    {POPI}
57: rd;
    sp := sp + 1;
```

```
69: pc := mbr;
    goto 0;
70: a := ac;          {SWAP}
    ac := sp;
    sp:= a;
    goto 0;
    en goto 76;
    {INSP}
    smask);
    a;
    {DESP}
    d(a, smask);
    a);
    a := a
    sp := sp + a;
    goto 0;
```

Micro-Programa Vertical

```
0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);
   if n then goto 11;
5: alu := tir;
   if n then goto 09;
6: mar := ir; rd;    {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;
```

```
23: alu := ac;        {IZER}
```

```
38: a := ir;          {SUBL}
```

```
58: a := mbr;
    mar := ac; mbr := a; wr;
    wr;
    goto 0;
    alu := tir;
    if n then goto 62;
    sp := sp + (-1); {PUSH}
    mar:= sp; mbr :=ac; wr;
    wr;
    goto 0;
    mar := sp; rd;    {POP}
    rd;
    sp := sp + 1;
64: ac := mbr;
    goto 0;
65: tir := lshift (tir ):
    if n then goto 73;
```

0: *mar* := *pc* ; *rd* ;		{ciclo principal}	
1: *pc* := *pc* + 1; *rd* ;		{incrementa pe}	
2: *ir* := *mbr* ; **if** *n* **then goto** 28;		{salva y decodifica mbr}	
3: *tir* := *lshift* (*ir* + *ir*); **if** *n* **then goto** 19;			
4: *tir* := *lshift* (*tir*); **if** *n* **then goto** 11;		{000x o 001x?}	
5: *alu* := *tir* ; **if** *n* **then goto** 9;		{0000 o 0001?}	
6: *mar* := *ir* ; *rd* ;		{0000 = LODD}	
7: *rd* ;			
8: *ac* := *mbr* ; **goto** 0;			
0110	**GETMBR**	Almacena MBR en registro	**r1:=MBR**
0100	**LSHIFT**	Desplaza a la Izquierda	**r1:=r2<<1**
1101	**NJUMP**	Salta si N=true	**if N then goto r1<<4 | r2**

```
29: tir:= lshift (tir);
    if n then goto 35;
30: alu := tir;
    if n then goto 33;
```

```
    if z then goto 0;
45: pc := ir;
    pc := band (pc, amask);
    goto 0;
```

```
    ac := ac + a;
    goto 0;
15: mar := ir; rd;    {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
    a := inv(a);
18: ac := ac + a;
    goto 0;
19: tir := Ishift(tir);
    if n then goto 25;
20: alu := tir;
    if n then goto 23;
21: alu := ac;        {JPOS}
    if n then goto 0;
22: pc := ir;
    pc band(pc, amask);
    goto 0;
```

```
34: mar := a; mbr := ac, wr;
    wr;
    goto 0;
35: alu := tir;
    if n then goto 38;
36: a := ir;          {ADDL}
    a := a + sp;
37: mar := a; rd;
    rd;
    a := mbr;
    ac := ac + a;
    goto 0;
```

```
50: tir := lshift (tir );
    if n then goto 55;
    tir := lshift
    then go
52: a
54:
    mar :=
    wr;
    goto
56: mar := sp; rd    {POPI}
57: rd;
    sp := sp + 1;
```

```
    ac := sp;
    sp:= a;
    goto 0;
    ;
    en goto 76;
    {INSP}
    smask);
    a;
    {DESP}
    d(a, smask);
    a);
    a := a
    sp := sp + a;
    goto 0;
```

Micro-Programa Vertical

Nano-Programación

Arquitectura de computadoras

Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta

Micro-Instrucciones repetidas

```
 0: mar := pc; rd;
 1: rd;
    pc := pc + 1;
 2: ir := mbr;
    tir := lshift(tir);
    if n then goto 28;
 3: tir := lshift(tir);
    if n then goto 19;
 4: tir := lshift(tir);
    if n then goto 11;
 5: alu := tir;
    if n then goto 09;
 6: mar := ir; rd;    {LODD}
 7: rd;
 8: ac := mbr;
    goto 0;
 9: mar := ir; mbr:= ac; wr;
                      {STOD}
10: wr
    goto 0;
11: alu := tir;
    if n then goto 15;
12: mar := ir; rd;    {ADDD}
13: rd;
14: a := mbr;
    ac := ac + a;
    goto 0;
15: mar := ir; rd;    {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
    a := inv(a);
18: ac := ac + a;
    goto 0;
19: tir := lshift(tir);
    if n then goto 25;
20: alu := tir;
    if n then goto 23;
21: alu := ac;        {JPOS}
    if n then goto 0;
22: pc := ir;
    pc band(pc, amask);
    goto 0;
    if n then goto 27;
26: pc:= ir;          {JUMP}
    pc:= band(pc, amask);
    goto 0;
27: ac := ir;         {LOCO}
    ac :=band(ac, amask);
    goto 0;
28: tir := lshift (tir);
    if n then goto 40;
29: tir := lshift (tir);
    if n then goto 35;
30: alu := tir;
    if n then goto 33;
31: a := ir;          {LODL}
    a := a + sp;
32: mar := a; rd;
    rd;
    ac := mbr;
    goto 0
33: a := ir;          {STOL}
    a := a + sp;
34: mar := a; mbr := ac, wr;
    wr;
    goto 0;
35: alu := tir;
    if n then goto 38;
36: a := ir;          {ADDL}
    a := a + sp;
37: mar := a; rd;
    rd;
    a := mbr;
    ac := ac + a;
    goto 0;
    goto 99;
40: tir := lshift(tir);
    if n then goto 46;
41: alu:= tir;
    if n then goto 44;
42: alu:= ac;         {JNEG}
    if n then goto 22;
43: goto 0;
44: alu := ac;        {JNZE}
    if z then goto 0;
45: pc := ir;
    pc := band (pc, amask);
    goto 0;
46: tir := lshift(tir);
    if n then goto 50;
47: sp := sp + (-1); {CALL}
48: mar := sp; mbr:= pc; wr;
    wr;
49: pc := ir;
    pc := band(pc, amask);
    goto 0;
50: tir := lshift (tir );
    if n then goto 65;
51: tir := lshift(tir);
    if n then goto 59;
52: alu := tir;
    if n then goto 56;
53: mar := ac; rd;    {PS
    rd;
54: sp := sp + (-1);
55: a := mbr;
    mar := sp; mbr:=
    wr;
    goto 0;
56: mar := sp; rd;    {POPI}
57: rd;
    sp := sp + 1;
 8: a := mbr;
    mar := ac; mbr := a; wr;
    wr;
    goto 0;
59: alu := tir;
    if n then goto 62;
60: sp := sp + (-1); {PUSH}
61: mar:= sp; mbr :=ac; wr;
    wr;
    goto 0;
62: mar := sp; rd;    {POP}
63: rd;
    sp := sp + 1;
64: ac := mbr;
    goto 0;
65: tir := lshift (tir );
    if n then goto 73;
66: alu := tir;
    if n then goto 70;
67: mar := sp; rd;    {RETN}
68: rd;
    sp := sp + 1;
69: pc := mbr;
    goto 0;
70: a := ac;          {SWAP}
71: ac := sp;
72: sp:= a;
    goto 0;
73: alu :=
    if n
    goto 76;
    :                 {INSP}
                      ask);
                      DESP}
    :                 1;
    sp :=             + a;
    goto 0;
```

Micro-Programa Vertical

Nano programación

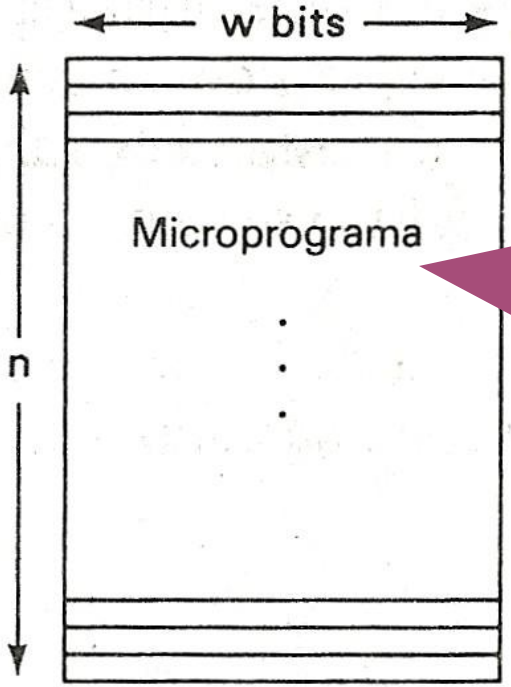
```
 0: mar := pc; rd;
 1: rd;
    pc := pc + 1;
 2: ir := mbr;
    tir := lshift(tir);
    if n then goto 28;
 3: tir := lshift(tir);
    if n then goto 19;
 4: tir := lshift(tir);
    if n then goto 11;
 5: alu := tir;
    if n then goto 09;
 6: mar := ir; rd;    {LODD}
 7: rd;
 8: ac := mbr;
    goto 0;
 9: mar := ir; mbr:= ac; wr;
                      {STOD}
10: wr
    goto 0;
11: alu := tir;
    if n then goto 15;
12: mar := ir; rd;    {ADDD}
13: rd;
14: a := mbr;
    ac := ac + a;
    goto 0;
15: mar := ir; rd;    {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
    a := inv(a);
18: ac := ac + a;
    goto 0;
19: tir := lshift(tir);
    if n then goto 25;
20: alu := tir;
    if n then goto 23;
21: alu := ac;        {JPOS}
    if n then goto 0;
22: pc := ir;
    pc band(pc, amask);
    goto 0;
```

```
    if n then goto 27;
26: pc:= ir;          {JUMP}
    pc:= band(pc, amask);
    goto 0;
27: ac := ir;         {LOCO}
    ac :=band(ac, amask);
    goto 0;
28: tir := lshift (tir);
    if n then goto 40;
29: tir := lshift (tir);
    if n then goto 35;
30: alu := tir;
    if n then goto 33;
31: a := ir;          {LODL}
    a := a + sp;
32: mar := a; rd;
    rd;
    ac := mbr;
    goto 0
33: a := ir;          {STOL}
    a := a + sp;
34: mar := a; mbr := ac, wr;
    wr;
    goto 0;
35: alu := tir;
    if n then goto 38;
36: a := ir;          {ADDL}
    a := a + sp;
37: mar := a; rd;
    rd;
    a := mbr;
    ac := ac + a;
    goto 0;
```

```
    goto 99;
40: tir := lshift(tir);
    if n then goto 46;
41: alu:= tir;
    if n then goto 44;
42: alu:= ac;         {JNEG}
    if n then goto 22;
43: goto 0;
44: alu := ac;        {JNZE}
    if z then goto 0;
45: pc := ir;
    pc := band (pc, amask);
    goto 0;
46: tir := lshift(tir);
    if n then goto 50;
47: sp := sp + (-1); {CALL}
48: mar := sp; mbr:= pc; wr;
    wr;
49: pc := ir;
    pc := band(pc, amask);
    goto 0;
50: tir := lshift (tir );
    if n then goto 65;
51: tir := lshift(tir);
    if n then goto 59;
52: alu := tir;
    if n then goto 56;
53: mar := ac; rd;    {PSHI}
    rd;
54: sp := sp + (-1);
55: a := mbr;
    mar := sp; mbr:=a, wr;
    wr;
    goto 0;
56: mar := sp; rd;    {POPI}
57: rd;
    sp := sp + 1;
```

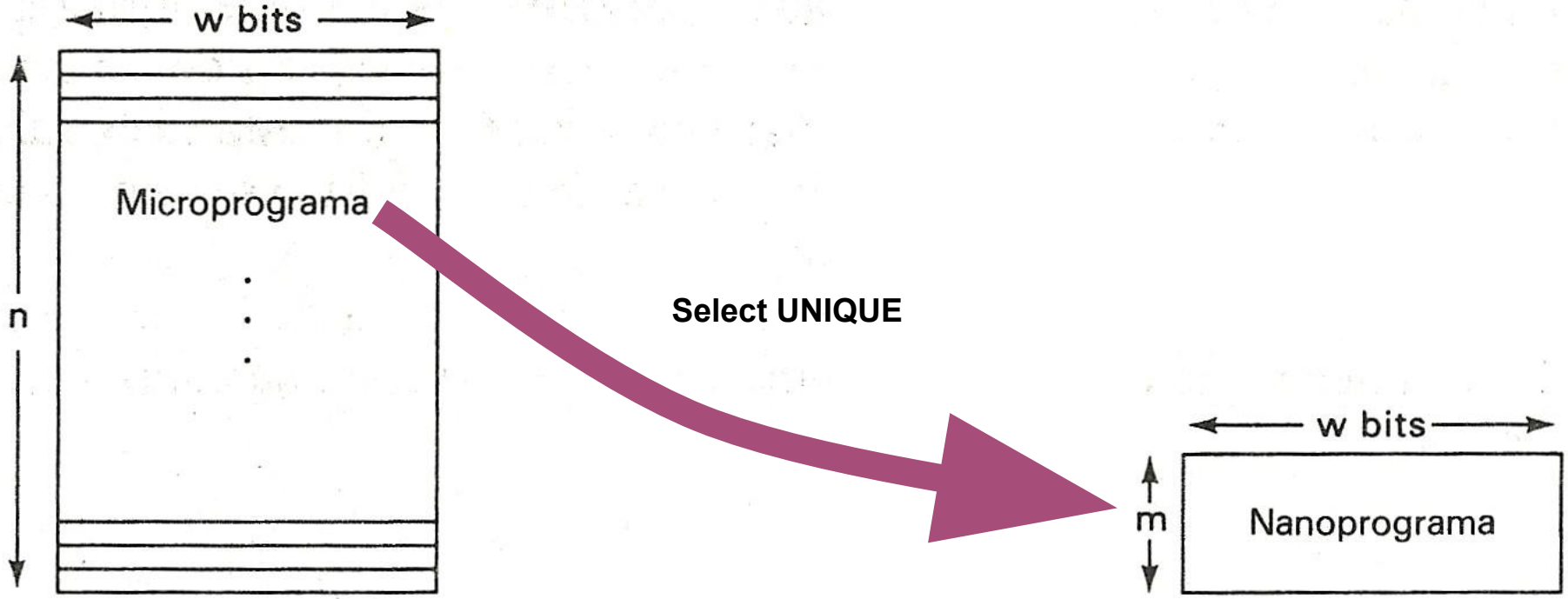
```
 8: a := mbr;
    mar := ac; mbr := a; wr;
    wr;
    goto 0;
59: alu := tir;
    if n then goto 62;
60: sp := sp + (-1); {PUSH}
61: mar:= sp; mbr :=ac; wr;
    wr;
    goto 0;
62: mar := sp; rd;    {POP}
63: rd;
    sp := sp + 1;
64: ac := mbr;
    goto 0;
65: tir := lshift (tir );
    if n then goto 73;
66: alu := tir;
    if n then goto 70;
67: mar := sp; rd;    {RETN}
68: rd;
    sp := sp + 1;
69: pc := mbr;
    goto 0;
70: a := ac;          {SWAP}
71: ac := sp;
72: sp:= a;
    goto 0;
73: alu := tir;
    if n then goto 76;
74: a := ir;          {INSP}
    a := band(a, smask);
75: sp := sp + a;
    goto 0;
76: a := ir;          {DESP}
    a := band(a, smask);
77: a := inv(a);
78: a := a + 1;
    sp := sp + a;
    goto 0;
```

Nano programación

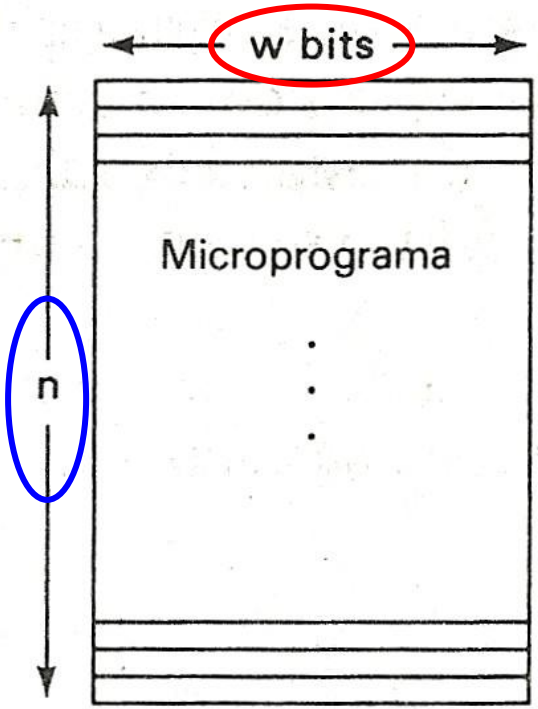


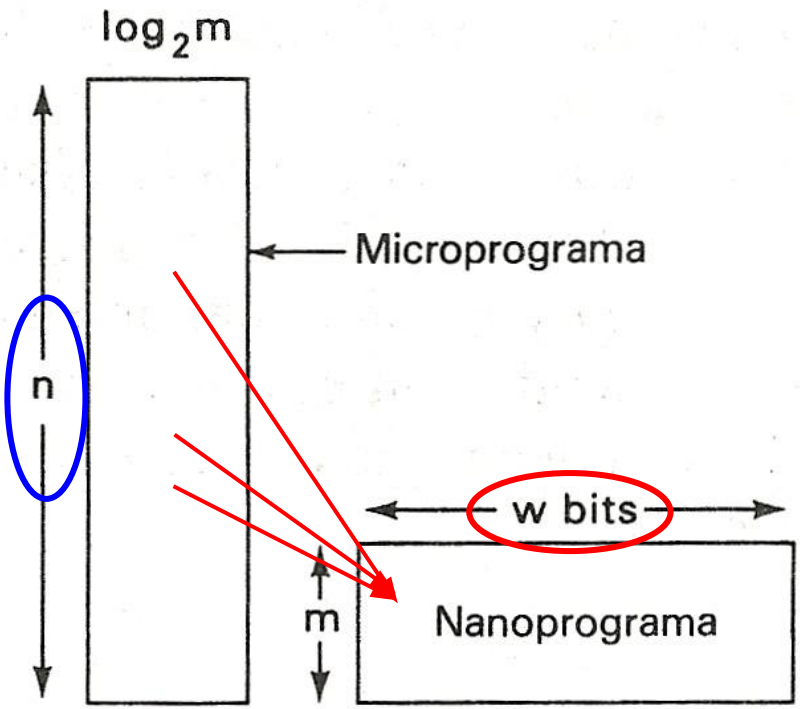
Muchas Micro-Instrucciones se repiten

Nano programación

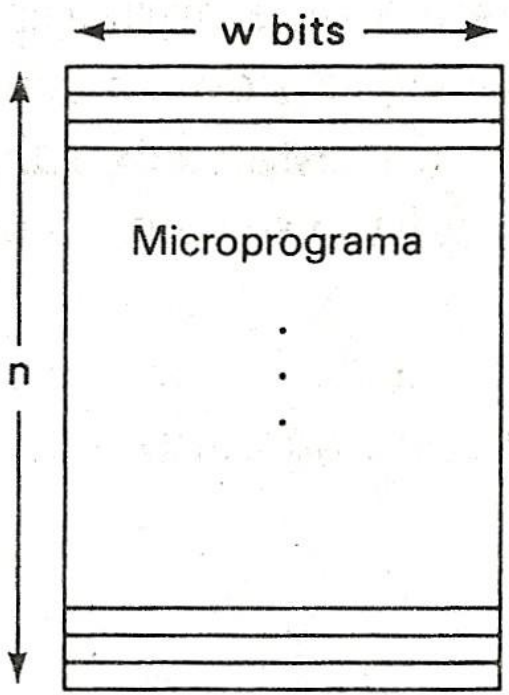


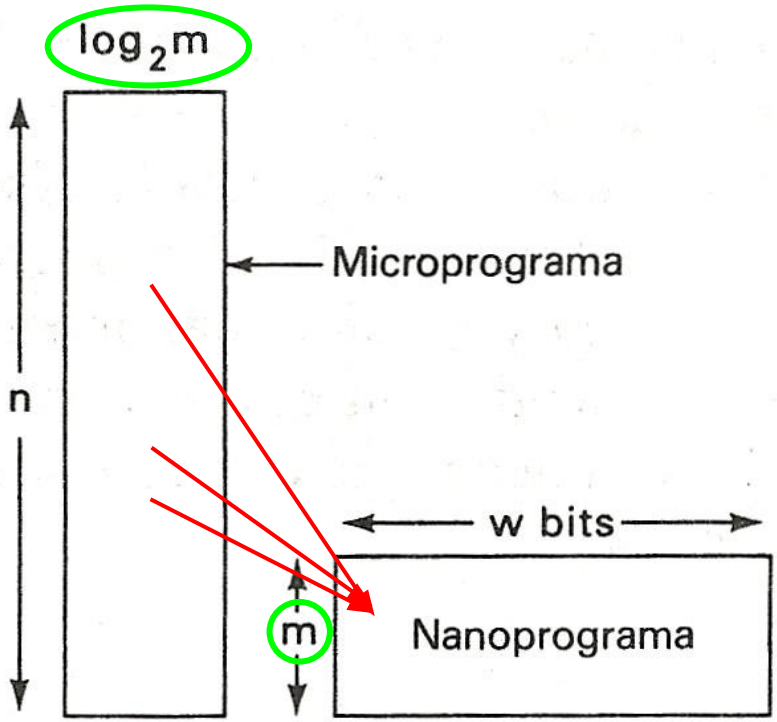
Nano memoria





Nano memoria





Pipe-Line

Arquitectura de computadoras

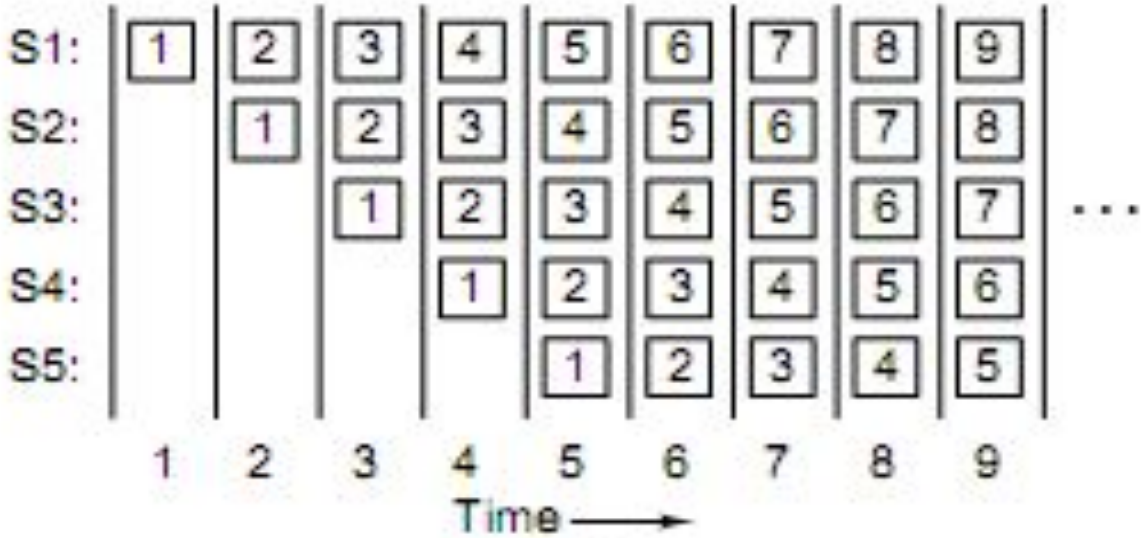
Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta

Pipe Line

- Mejora del rendimiento
- Varias unidades específicas

Pipe Line

- Mejora del rendimiento
- Varias unidades específicas



Pipe Line

- Mejora del rendimiento
- Varias unidades específicas
- 30% de las instrucciones son ***saltos***
  - ***incondicionales***
  - ***condicionales***
  - ***iteraciones***

Pipe Line

- Mejora del rendimiento
- Varias unidades específicas
- 30% de las instrucciones son ***saltos***

	1	2	3	4	5	6	7	8	9	10	11		
Extracción de instrucciones	1	2	B					4	5	6	7	8	9
Decodificación de instrucciones		1	2	B					4	5	6	7	8
Cálculo de direcciones			1	2	B					4	5	6	7
Extracción de operandos				1	2	B					4	5	6
Ejecución					1	2	B					4	5

Pipe Line

Los compiladores optimizan el ejecutable a fin de minimizar la Penalización por Salto

	1	2	3	4	5	6	7	8	9	10	11		
Extracción de instrucciones	1	2	B					4	5	6	7	8	9
Decodificación de instrucciones		1	2	B					4	5	6	7	8
Cálculo de direcciones			1	2	B					4	5	6	7
Extracción de operandos				1	2	B					4	5	6
Ejecución					1	2	B					4	5

PENALIZACIÓN POR SALTO

Micro Arquitectura Vertical

Nano-Programación y Pipe-Line

Arquitectura de computadoras

Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta